

MANUAL TÉCNICO

1. INTRODUCCIÓN Y ARQUITECTURA GENERAL

Este proyecto consiste en la implementación de un entorno de desarrollo, compilación y ejecución para un subconjunto del lenguaje Go, denominado "GoLite". El sistema está estructurado bajo el patrón clásico de fases de un compilador/intérprete, dividiéndose de la siguiente manera:

- Analizador Léxico (Lexer): Tokeniza el flujo de caracteres de entrada.
- Analizador Sintáctico (Parser): Construye el Árbol de Sintaxis Abstracta (AST) a partir de los tokens utilizando CUP (java-cup-11b.jar).
- Analizador Semántico e Intérprete: Recorre el AST mediante el patrón Visitor para validar tipos y ejecutar directamente la función 'main'.
- Interfaz Gráfica de Usuario (GUI): Desarrollada en Swing (NetBeans), permitiendo la edición de código, visualización de tokens y errores.

2. COMPONENTE LÉXICO (LEXER) Y ESTRUCTURA DE TOKENS

El analizador léxico identifica las palabras clave, identificadores, operadores y literales del lenguaje. Cada elemento reconocido se mapea a una instancia de la clase 'Token'.

Estructura de la clase Token (com.mynor.golite.lexer.Token):

- linea (int): Ubicación física de la fila en el editor.
- columna (int): Ubicación física del carácter en la fila.
- lexema (String): El texto plano reconocido (ej. "if", "sum", "45.2").
- tipo (TipoToken): Un enumerador que clasifica la categoría del token.

Visualización:

Para facilitar la depuración, la GUI implementa un JTable que renderiza dinámicamente el listado de tokens recuperados con sus cuatro propiedades mediante un DefaultTableModel no editable, desplegado en un JDialog.

3. ANALIZADOR SEMÁNTICO E INTÉRPRETE

Características principales de la implementación:

- Tabla de Símbolos: Representada por un único 'Map<String, Object>' que asocia directamente el identificador de la variable con su valor nativo de Java (Integer, Double, String, Boolean o List para slices).
- Ámbito Único: Diseñado exclusivamente para procesar y ejecutar de forma lineal las instrucciones de la función raíz 'main'.
- Manejo de Slices: Soporta la instanciación de arreglos dinámicos utilizando 'ArrayList<Object>' de Java, implementando operaciones como longitud (len) y adición de elementos (append).

4. INTERFAZ GRÁFICA Y CONTROL DE ERRORES

El sistema cuenta con mecanismos de reporte visual de errores léxicos, sintácticos y semánticos.

Manejo de Errores:

- Los errores semánticos se recopilan en una estructura 'ArrayList<String[]>' donde cada fila contiene [Tipo de Error, Descripción / Ubicación].
- Al igual que los tokens, se presentan al desarrollador por medio de un componente dinámico JTable, asegurando un aislamiento total de las celdas para prevenir modificaciones accidentales por parte del usuario.